

Minigolf fisico 2D con Pymunk

Salma Boulagna Moreno

Resumen

Este proyecto consiste en un minigolf 2D desarrollado con Pymunk y Pygame. La simulación se plantea en vista cenital para centrar el modelo físico en la rodadura, el impulso inicial, las colisiones, la pérdida de energía y la influencia de distintas superficies. El juego incluye cinco hoyos organizados como niveles independientes, cada uno con posición inicial, hoyo, obstáculos y zonas de terreno propias.

Índice

1. Objetivo del proyecto	2
2. Funcionamiento general	2
3. Modelo físico	2
3.1. Sistema de unidades	2
3.2. Bola de golf	2
3.3. Golpe como impulso	3
3.4. Resistencia a la rodadura	3
3.5. Superficies	4
3.6. Criterio de entrada en el hoyo	4
3.7. Velocidad angular de la bola	4
4. Texturas	5

1. Objetivo del proyecto

El objetivo es construir un prototipo jugable de minigolf que muestre de forma clara varios conceptos de dinamica aplicados a videojuegos. El jugador golpea la bola mediante un gesto de arrastre con el raton, y el sistema traduce ese gesto en un impulso fisico aplicado sobre el cuerpo.

La eleccion de una vista cenital simplifica el problema respecto al golf real. En lugar de simular el vuelo de la bola, el loft del palo o la aerodinamica, el proyecto se centra en el tipo de movimiento dominante en un minigolf: desplazamiento sobre el suelo, rozamiento por rodadura, rebotes y control de precision.

2. Funcionamiento general

Cada hoyo ocupa una sola pantalla y contiene una bola, un agujero, zonas de superficie y obstaculos. El usuario debe embocar la bola con el menor numero de golpes posible.

Los controles principales son:

- Mantener pulsado el boton izquierdo del raton cerca de la bola para apuntar.
- Arrastrar en direccion contraria al golpe deseado para aumentar la potencia.
- Soltar el boton del raton para aplicar el golpe.
- Pulsar R para reiniciar el hoyo actual.
- Pulsar N para pasar al siguiente hoyo.
- Pulsar ESC para salir.

El marcador muestra el hoyo actual, el numero de golpes, la superficie sobre la que esta la bola, el coeficiente de rodadura de dicha superficie y la velocidad instantanea de la bola.

3. Modelo fisico

3.1. Sistema de unidades

La simulacion usa metros, kilogramos y segundos para los calculos fisicos. La pantalla, sin embargo, se dibuja en pixeles. Para relacionar ambos sistemas se define una escala:

$$1 \text{ m} = 360 \text{ px.}$$

De este modo, Pymunk trabaja con magnitudes fisicas coherentes y Pygame solo se encarga de convertirlas a pixeles para dibujar.

3.2. Bola de golf

La bola se modela como un cuerpo dinamico circular de Pymunk. Se utiliza una masa aproximada de bola de golf real:

$$m = 0,04593 \text{ kg,}$$

y un radio:

$$R = 0,02134 \text{ m.}$$

Pymunk necesita conocer tanto la masa como el momento de inercia. Por eso el codigo calcula el momento mediante:

```
moment = pymunk.moment_for_circle(BALL_MASS, 0.0, BALL_RADIUS)
self.body = pymunk.Body(BALL_MASS, moment)
```

La forma de colision se define con `pymunk.Circle`, asociada al cuerpo dinamico de la bola.

3.3. Golpe como impulso

El golpe del jugador se representa mediante un impulso aplicado en el centro de la bola. El vector de golpe se obtiene a partir del arrastre del raton. Si el usuario arrastra mas, el impulso aumenta hasta un limite maximo.

La relacion usada es proporcional:

$$J = J_{max} \frac{d}{d_{max}},$$

con J como impulso aplicado, J_{max} como impulso maximo, d como distancia de arrastre y d_{max} como distancia maxima considerada.

En el codigo se aplica asi:

```
impulse = direction.normalized() * min(MAX_IMPULSE, max(0.0, strength))
self.body.apply_impulse_at_world_point(impulse, self.body.position)
```

Al aplicarse en el centro de masas, el impulso produce principalmente movimiento lineal. Esto evita introducir una rotacion artificial causada por un golpe descentrado.

3.4. Resistencia a la rodadura

Cada superficie tiene un coeficiente de rodadura C_{rr} . La fuerza de frenado se modela como:

$$F_r = C_{rr}N.$$

Como el campo es horizontal, la normal se aproxima por:

$$N = mg.$$

Por tanto:

$$F_r = C_{rr}mg.$$

La fuerza siempre se aplica en sentido contrario a la velocidad de la bola:

$$\vec{F}_r = -F_r \hat{v}.$$

Para evitar errores numericos, la fuerza de frenado se limita al valor necesario para detener la bola durante el paso de simulacion:

$$F_{stop} = \frac{m\|\vec{v}\|}{\Delta t}.$$

La fuerza aplicada finalmente es:

$$F_{aplicada} = \min(F_r, F_{stop}).$$

Esto evita que el rozamiento invierta artificialmente el sentido de la velocidad en un solo paso.

Superficie	Coeficiente C_{rr}	Efecto en la bola
Green	0.12	La bola rueda con facilidad.
Calle	0.25	Frenado intermedio.
Hierba alta	0.55	La bola pierde velocidad rapidamente.
Arena	0.90	La bola se frena mucho.
Agua	-	Penalizacion y reinicio.

Cuadro 1: Superficies usadas en el minigolf.

3.5. Superficies

El campo se divide en zonas rectangulares. Cada zona tiene un color, una textura y un coeficiente de rodadura. Si la bola esta dentro de una zona, se usa el coeficiente de esa superficie para calcular el frenado.

El agua no se trata como una superficie normal. Si la bola entra en una zona de agua, se suma una penalizacion y la bola vuelve a la posicion inicial del hoyo.

3.6. Criterio de entrada en el hoyo

La entrada en el hoyo se decide mediante dos condiciones. Primero se calcula la distancia entre el centro de la bola y el centro del hoyo:

$$d = \|\vec{r}_{bola} - \vec{r}_{hoyo}\|.$$

La bola solo puede entrar si su centro esta dentro de un radio de captura:

$$d \leq R_{hoyo} k_{captura}.$$

En el proyecto se usa:

$$k_{captura} = 0,72.$$

Ademas, se exige que la velocidad de la bola sea menor que una velocidad maxima de captura:

$$\|\vec{v}\| \leq v_{captura}.$$

En el codigo se usa:

$$v_{captura} = 0,85 \text{ m/s}.$$

Este criterio evita que una bola que cruza muy rapido por encima del hoyo sea embocada de manera poco realista. Por tanto, no basta con tocar visualmente el agujero: la bola debe estar suficientemente centrada y moverse a una velocidad compatible con caer dentro.

3.7. Velocidad angular de la bola

La velocidad angular se utiliza principalmente como recurso visual para representar la rodadura. En una rodadura ideal se cumple:

$$v = \omega R,$$

por lo que:

$$\omega = \frac{v}{R}.$$

En cada actualizacion se calcula una velocidad angular objetivo a partir de la velocidad lineal de la bola. Luego se aproxima suavemente el valor actual hacia ese objetivo para evitar saltos visuales bruscos:

```
target_omega = speed / BALL_RADIUS
self.body.angular_velocity = (
    (1.0 - ROLL_SPIN_SMOOTHING) * self.body.angular_velocity
    + ROLL_SPIN_SMOOTHING * sign * target_omega
)
```

Esta rotacion no gobierna la trayectoria de la bola. La trayectoria depende del impulso inicial, las colisiones y la resistencia a la rodadura. La velocidad angular se usa para que la marca dibujada sobre la bola gire y transmita visualmente que la bola esta rodando.

4. Texturas

Las superficies del campo se dibujan con texturas PNG repetidas sobre cada zona rectangular. Si alguna textura no se encuentra, el codigo mantiene un color plano como respaldo. Esta decision permite que el proyecto siga siendo ejecutable aunque falte algun archivo visual.

Las texturas principales incluidas son:

- `green.png`: green del campo.
- `fairway.png`: calle.
- `rough.png`: hierba alta.
- `sand.png`: arena fotografica y granulada.
- `water.png`: agua.
- `wood.png`: obstaculos.